

# AIAA 3111 - 数据挖掘导论

## 第 4 讲：模式挖掘：高级方法

Dr. Zhidan Liu

香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

# 第 3 讲回顾：我们挖掘什么

| 概念 | 所回答的问题 | 关键条件 |
| --- | --- | --- |
| 频繁项集 | 哪些项会同时出现? | 支持度 ⩾ minsup |
| 关联规则 | 给定 X， Y 有多常随之出现? | 置信度与提升度 |
| 闭模式 | 扩展后支持度能否保持? | 无等支持度超集 |
| 最大模式 | 模式扩展后能否仍频繁? | 无频繁超集 |

**支持度衡量普遍程度；置信度是条件性的；提升度将规则与独立性比较。**

# 模式挖掘：高级方法

- 挖掘多层与多维模式
- 基于约束的模式挖掘
- 挖掘序列模式
- 挖掘子图模式
- 小结

# 第 4 讲路线图

## 第 1 部分：更广泛的模式与约束

- 多层与多维关联
- 稀有模式与负相关模式
- 约束性质与剪枝策略

## 第 2 部分：有序模式与结构化模式

- 序列模式与 GSP
- 基于投影数据库的模式增长
- 频繁子图与闭子图挖掘

**第 3 讲聚焦频繁项集；**
**第 4 讲扩展模式语言与搜索空间。**

# 为什么需要高级模式挖掘？

## 基本购物篮假设

- 单一扁平的项空间
- 单一支持度阈值
- 无序事务
- 挖掘全部合格模式

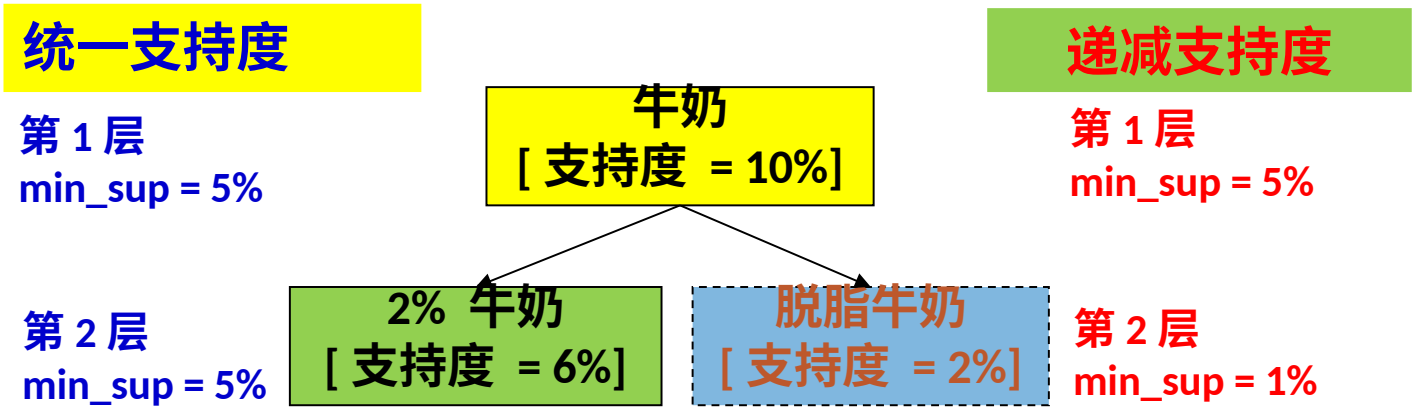
## 真实数据

- 项属于层次与维度
- *稀有事件可能价值很高*
- 顺序与图结构可能重要
- 用户常常指定约束

**高级方法改变模式表示、兴趣度度量或搜索策略。**

# 挖掘多层频繁模式

- 项常常形成层次
  - 牛奶： Dairyland 2% 牛奶
  - 面包： Wonder 小麦面包




- 遵循概念层次的项的最小支持度阈值
  - **多层统一最小支持度**
  - **层次递减最小支持度：较低层的项预期支持度更低**
- 高效挖掘：共享多层挖掘
  - 用最低最小支持度向下传递候选集

# 挖掘多层关联时的冗余过滤

- **冗余过滤：因“祖先”关系而冗余**
  - 规则 (1)： 牛奶 ⇒ 小麦面包 [ 支持度 = 8%， 置信度 = 70%]
  - 规则 (2)： 2% 牛奶 ⇒ 小麦面包 [ 支持度 = 2%， 置信度 = 72%]
    - 设售出的 2% 牛奶约占按加仑计牛奶销量的 ¼
      - 规则 (2) 很大程度上可由规则 (1) 及 2% 牛奶的普遍程度解释。

- 一般而言：若某规则的支持度接近按其“祖先”规则得到的“期望”值，且置信度与其“祖先”相近，则该规则冗余

问：若规则 (1) 是规则 (2) 的祖先，应剪掉哪一条？

# 概念层次间的冗余

**祖先信息为更具体的规则提供基线。**

1. 祖先规则牛奶 ⇒ 小麦面包的支持度为 8% ，置信度为 70% 。
2. 设 2% 牛奶占全部牛奶销量的四分之一。
3. 2% 牛奶 ⇒ 小麦面包的期望支持度约为 8% × 1/4 = 2% 。
4. 观测支持度 2% 与置信度 72% 相对祖先规则几乎不增加信息。

Rule (1): milk ⇒ wheat bread [support = 8%, confidence = 70%]

Rule (2): 2% milk ⇒ wheat bread [support = 2%, confidence = 72%]

**当后裔规则明显偏离从祖先继承的期望时，它才有趣。**

# 为不同种类的项定制最小支持度

- 在每次关联挖掘中，我们对所有待挖掘的项或项集使用同一最小支持度阈值
- 现实中，有些项（如钻石、手表等）价值高但出现较少
- 因此需要为不同种类的项设置定制的最小支持度

- 一种方法：使用基于分组的“个性化”最小支持度
  - 例如，{ 钻石，手表 }： 0.05%；{ 面包，牛奶 }： 5%；…
  - 如何高效挖掘这类规则?
    - 现有可扩展挖掘算法可以很容易扩展以覆盖这类情形

# 挖掘多维关联

- **单维规则（例如，项都在“商品”维）**
  - buys(X, "milk") ⇒ buys(X, "bread")
- **多维规则（即项位于 ≥ 2 个维或谓词中）**
  - 维间关联规则（谓词不重复）
    - age(X, "18-25") ∧ occupation(X, "student") ⇒ buys(X, "coke")
  - 混合维关联规则（谓词可重复）
    - age(X, "18-25") ∧ buys(X, "popcorn") ⇒ buys(X, "coke")
- 属性可为类别型或数值型
  - 类别属性（如职业、商品：取值之间无序）：用数据立方体做维间关联
  - 数值属性：数值型，取值间有隐式顺序——离散化、聚类与梯度方法

# 关联规则的三种形式

| 规则类型 | 谓词 | 示例 |
| --- | --- | --- |
| 单维 | 一个重复的维 | buys(milk) ⇒ buys(bread) |
| 维间 | 无重复谓词 | age(18–25) ∧ student ⇒ buys(coke) |
| 混合维 | 至少一个重复谓词 | age(18–25) ∧ buys(popcorn) ⇒ buys(coke) |

- **维间规则：每个谓词至多出现一次**
- **混合维规则：至少一个谓词出现多于一次**

# 稀有模式与负模式

- **稀有模式**
  - 支持度极低但有趣（例如购买劳力士手表）
  - 如何挖掘？为不同组项设置个性化、基于分组的最小支持度阈值
- **负模式**
  - 负相关：不太可能同时发生
  - 例：同一顾客不太可能既买 Ford Expedition （一款 SUV ）又买 Ford Fusion （一款混合动力车），因此购买二者很可能是负相关模式
  - 如何定义负模式?

# 定义负相关模式

- 基于支持度的定义
  - 若项集 A 与 B 都频繁，但很少同时出现，即
    - $sup(A \cup B) << sup(A) \times sup(B)$
  - 则 A 与 B 负相关
- 这对大型事务数据集是好定义吗?
  - 例：设商店两种针线包 A 与 B 各售出 100 次，但只有一笔事务同时包含 A 与 B
    - 当事务总数为 200 时，有
      - $s(A \cup B) = 0.005, s(A) \times s(B) = 0.25, s(A \cup B) << s(A) \times s(B)$
    - 加入 19,800 笔零事务后（N = 20,000）：
      - $s(A \cup B) = 0.00005, s(A) \times s(B) = 0.000025$，于是结论反转
- 这里的问题是什么?

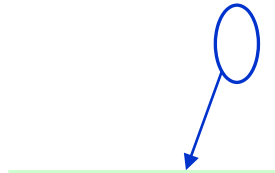
  **零事务会改变结果：该定义不是零事务不变的**

这是否让你想起提升度的定义?

# 定义负相关：定义中需要零事务不变性

- 有用的负相关度量应当是零事务不变的
  - 直觉：两个项集 A 与 B 是否负相关，不应受零事务数量的影响
- 基于 Kulczynski 度量的定义
  - 若项集 A 与 B 频繁，但 $(s(A \cup B)/s(A) + s(A \cup B)/s(B))/2 < \epsilon$，则 A 与 B 负相关
- 对同一针线包问题：
  - 该值在 N = 200 与 N = 20,000 时不变
    - 若 $\varepsilon = 0.02$，则有

$$Kulc(A,B) = (0.01 + 0.01)/2 = 0.01 < \varepsilon$$



$\epsilon$ 是负
模式阈值

# 快速检查：哪种度量是零事务不变的?

**问题：加入 10,000 笔既不含 A 也不含 B 的事务。哪种度量保持不变?**

| | |
|---|---|
| A. 相对支持度 (A) | B. 提升度 (A,B) |
| C. Kulc(A,B) | D. 相对支持度 (A∪B) |

**答案： C. Kulc(A,B)**

# 为何进行基于约束的挖掘?

- 实践中的模式挖掘：常常是用户引导的交互过程
  - 用户用数据挖掘查询语言（或图形界面）指定挖掘内容，并给出各类约束
- 基于约束的挖掘：连同用户给出的约束一起挖掘
- 为何要基于约束挖掘?
  - 用户灵活性：用户对挖掘内容给出约束
  - 优化：系统利用这些约束提高挖掘效率
    - 例如，将约束深入推进到挖掘过程中

# 基于约束的挖掘流程

**同一约束既可以指导结果，也可以指导搜索。**

1. 用户指定目标数据、模式形式和兴趣度条件。
2. 系统将查询转换为对模式和数据的约束。
3. 挖掘算法把可安全使用的约束推进到候选生成或投影中。
4. 系统检验剩余候选，只返回合格模式。

**只有系统能在最终输出之前实施约束，该约束才能提高效率。**

# 模式挖掘中的用户约束

| 约束类型 | 示例 | 控制 |
| --- | --- | --- |
| **数据** | 今年的纽约门店 | 哪些记录进入挖掘 |
| **维 / 层** | 品牌与顾客类别 | 哪些属性出现 |
| **兴趣度** | 支持度 ≥ 0.02 | 哪些模式合格 |
| **模式形式** | 包含牛奶；长度 ≤ 4 | 模式必须呈现的形式 |

**本讲聚焦模式约束，因为其数学性质可用于剪枝。**

# 用模式反单调性剪枝模式空间

| TID | 事务 |
|---|---|
| 10 | a, b, c, d, f, h |
| 20 | b, c, d, f, g, h |
| 30 | b, c, d, f, g |
| 40 | a, c, e, f, g |

min_sup = 2

| 项 | 价格 | 利润 |
|---|---|---|
| a | 100 | 40 |
| b | 40 | 0 |
| c | 150 | −20 |
| d | 35 | −15 |
| e | 55 | −30 |
| f | 45 | −10 |
| g | 80 | 20 |
| h | 10 | 5 |

- 约束 c 是反单调的
  - 若项集 S 违反约束 c，则其任意超集也违反
  - 因此，对项集 S 的挖掘可以终止
- 例 1： c1： sum(S.price) ≤ v 是反单调的
- 例 2： c2： range(S.profit) ≤ 15 是反单调的
  - 项集 ab 违反 c2 （range(ab) = 40）
  - ab 的每个超集也都违反

$$range(S.profit) = \max(S.profit) - \min(S.profit)$$

- 例 3. c3： sum(S.Price) ≥ v 不是反单调的
- 例 4. c4： support(S) ≥ σ 是反单调的吗?
  - 是！ Apriori 剪枝本质上就是用反单调约束进行剪枝!

注： item.price > 0
利润可以为负

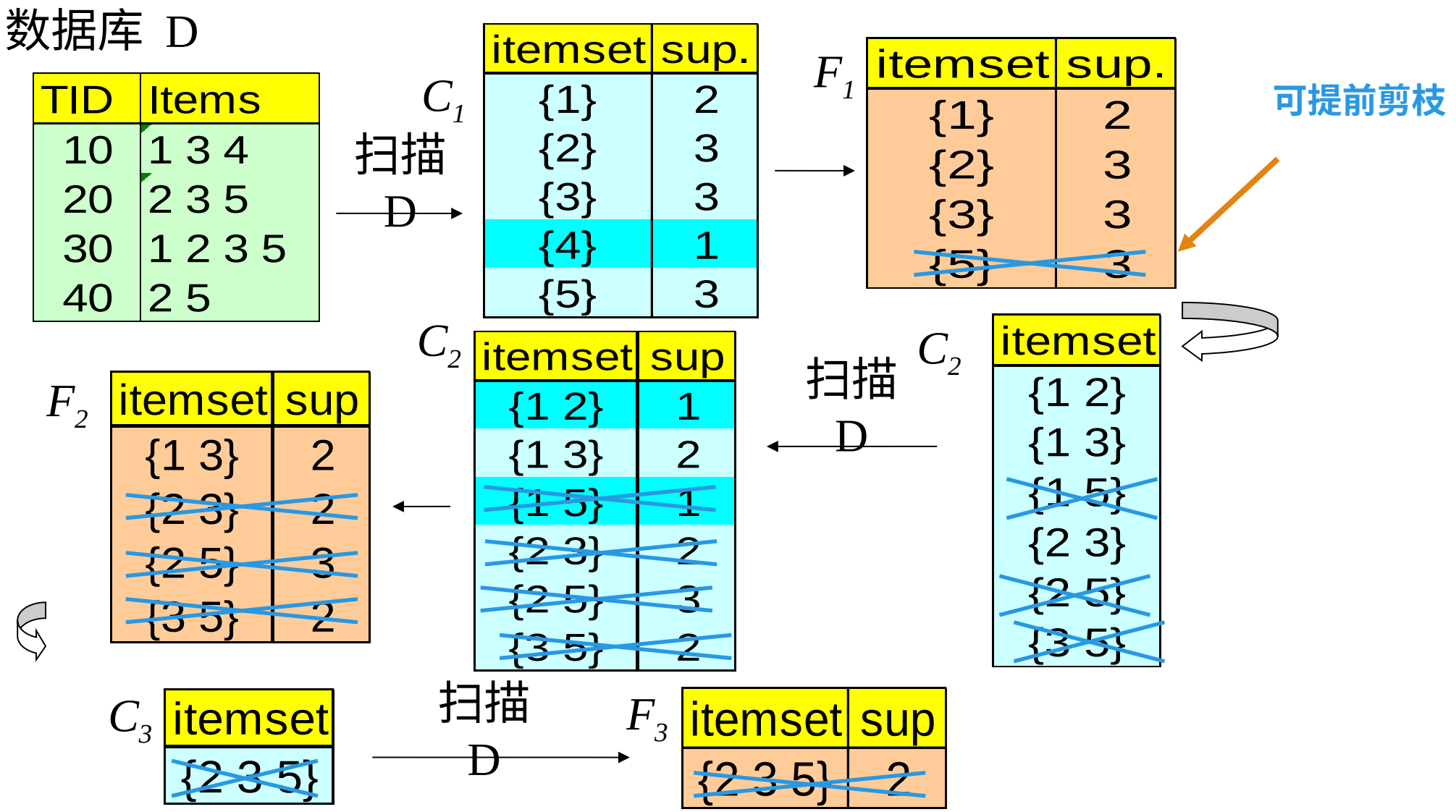
# 面向模式反单调约束的 Apriori

| 项 | 价格 |
|---|---|
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |
| 4 | 4 |
| 5 | 5 |

Min_sup=2

约束：

Sum{S.price} < 5



# 快速检查：是否反单调?

**问题：假设所有价格为正。哪个约束是反单调的?**

| | |
|---|---|
| A. sum(price) ≥ 100 | B. average(profit) ≥ 20 |
| C. sum(price) ≤ 100 | D. length ≥ 3 |

**答案： C. sum(price) ≤ 100**

# 模式单调性及其作用

| TID | 事务 |
|---|---|
| 10 | a, b, c, d, f, h |
| 20 | b, c, d, f, g, h |
| 30 | b, c, d, f, g |
| 40 | a, c, e, f, g |

min_sup = 2

| 项 | 价格 | 利润 |
|---|---|---|
| a | 100 | 40 |
| b | 40 | 0 |
| c | 150 | −20 |
| d | 35 | −15 |
| e | 55 | −30 |
| f | 45 | −10 |
| g | 80 | 20 |
| h | 10 | 5 |

- 约束 c 为单调：若项集 S 满足约束 c，则其任一超集也满足
  - 即后续挖掘中无需再检查 c
  - 收益不如反单调
- 例 1： c1: sum(S.Price) ≥ v 为单调
- 例 2： c2: min(S.Price) ≤ v 为单调
- 例 3： c3: range(S.profit) ≥ 15 为单调
  - 项集 ab 满足 c3
  - ab 的每个超集也满足

# 单调约束与反单调约束

| 性质 | 反单调 | 单调 |
| --- | --- | --- |
| **继承** | 不满足传递给超集 | 满足传递给超集 |
| **搜索用途** | 失败后剪枝 | 满足后停止复查 |
| **价格为正** | sum(price) ≤ v | sum(price) ≥ v |
| **支持度** | support ≥ σ | support ≤ σ |

**反单调约束通常在前缀增长搜索中提供更强的早期剪枝。**

# 可转换约束：对事务中的数据排序

| TID | 事务 |
|---|---|
| 10 | a, b, c, d, f, h |
| 20 | a, b, c, d, f, g, h |
| 30 | b, c, d, f, g |
| 40 | a, c, e, f, g |

min_sup = 2

| 项 | 价格 | 利润 |
|---|---|---|
| a | 100 | 40 |
| b | 40 | 0 |
| c | 150 | −20 |
| d | 35 | −15 |
| e | 55 | −30 |
| f | 45 | −5 |
| g | 80 | 30 |
| h | 10 | 5 |

- 通过对事务中的项适当排序，将困难约束转换为（反）单调

- 考察 c1: avg(S.profit) > 20
  - 按（profit）值降序排列各项
    - <*a*, *g*, *h*, *b*, *f*, *d*, *c*, *e*>
  - 项集 ab 违反 c1 （ avg(ab) = 20 ）
    - ab* （即 ab 投影数据库）同样违反
    - 因此 c1 并非全局反单调，但在利润降序下为可转换反单调。

# 可转换约束依赖于增长顺序

- 平均利润在任意集合扩展下既非单调也非反单调
- 按利润降序排列各项，且只增长遵守顺序的前缀
- 当前缀平均值低于阈值后，后续更低利润的项无法将其恢复
- 此剪枝对前缀增长有效，但对任意 Apriori 子集生成无效

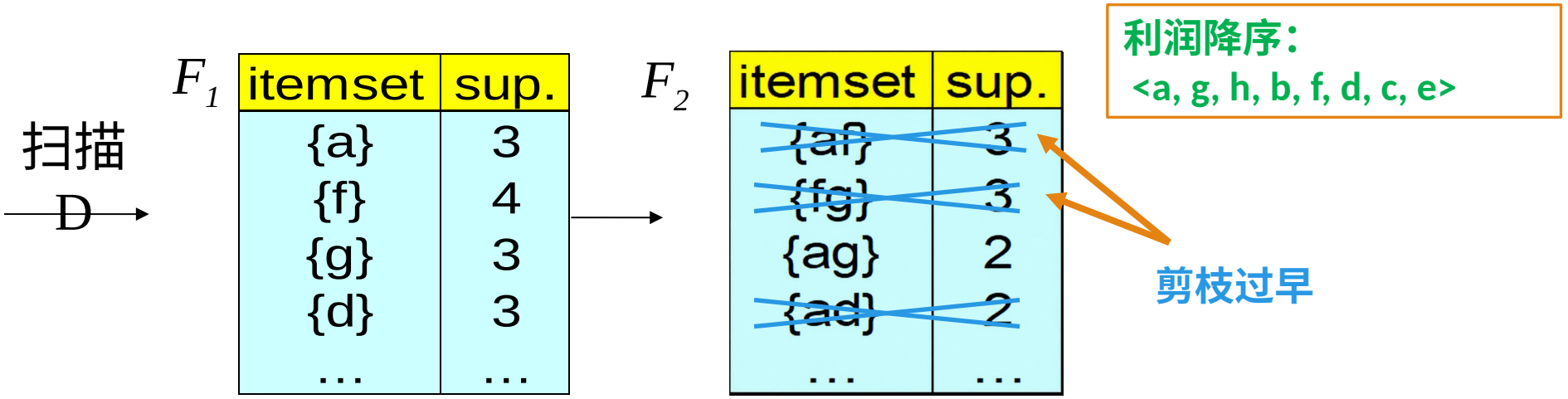
**排序规则是证明的一部分。没有它，剪枝判定就不安全。**

# 项重排序对 Apriori 有效吗?

| TID | 事务 |
|---|---|
| 10 | a, b, c, d, f, h |
| 20 | a, b, c, d, f, g, h |
| 30 | b, c, d, f, g |
| 40 | a, c, e, f, g |

| 项 | 价格 | 利润 |
|---|---|---|
| a | 100 | 40 |
| b | 40 | 0 |
| c | 150 | −20 |
| d | 35 | −15 |
| e | 55 | −30 |
| f | 45 | −5 |
| g | 80 | 30 |
| h | 10 | 5 |

**Min_sup=2; 约束： avg(S.profit) > 20**




- avg(*gf*) = 12.5 < 20, avg(af) = 17.5 < 20, avg(ag) = 35 > 20
- 但 avg(agf) = 21.7 > 20
- Apriori 不会将 "agf" 生成为候选

**重排序使前缀剪枝安全，但不能使 Apriori 的全子集剪枝安全。**

# 基于数据反单调性的数据空间剪枝

| TID | 事务 |
|---|---|
| 10 | a, b, c, d, f, h |
| 20 | b, c, d, f, g, h |
| 30 | b, c, d, f, g |
| 40 | a, c, e, f, g |

min_sup = 2

| 项 | 价格 | 利润 |
|---|---|---|
| a | 100 | 40 |
| b | 40 | 0 |
| c | 150 | −20 |
| d | 35 | −15 |
| e | 55 | −30 |
| f | 45 | −10 |
| g | 80 | 20 |
| h | 10 | 5 |

- 约束 c 为数据反单调：在挖掘过程中，若数据条目 t 在 c 下不能满足模式 p，则 t 也不能满足 p 的超集
  - 数据空间剪枝：可剪去数据条目 t
- 例 1： c1: sum(S.Profit) ≥ v 为数据反单调
  - 设约束 c1 为： sum(S.Profit) ≥ 25
    - T30: {b, c, d, f, g} 可删除，因其任意组合都无法构成利润之和 ≥ 25 的 S
- 例 2： c2: min(S.Price) ≤ v 为数据反单调
  - 考虑 v = 5，但某事务（如 T50）中每一项的价格都高于 10
- 例 3： c3: range(S.Profit) > 25 为数据反单调

# 数据空间剪枝应递归探索

| TID | 事务 |
|---|---|
| 10 | a, b, c, d, f, h |
| 20 | b, c, d, f, g, h |
| 30 | b, c, d, f, g |
| 40 | a, c, e, f, g |

min_sup = 2

| 项 | 利润 |
|---|---|
| a | 40 |
| b | 0 |
| c | −20 |
| d | −15 |
| e | −30 |
| f | −10 |
| g | 20 |
| h | 5 |

- 例 . c3: range(S.Profit) > 25
  - 检查 b 的投影数据库
    - 但项 "a" 不频繁（ sup = 1 ）

b 的投影数据库

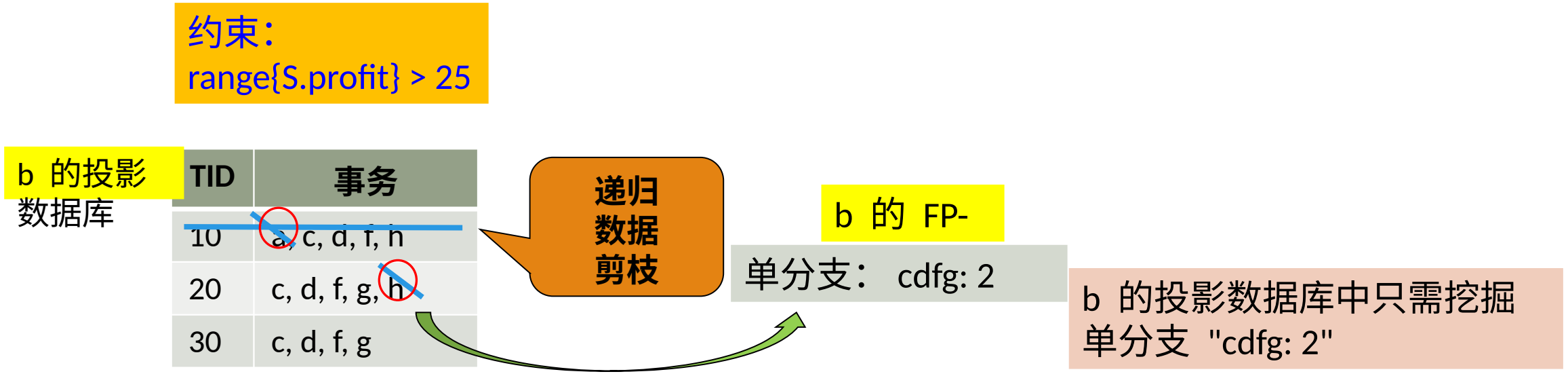
| TID | 事务 |
|---|---|
| 10 | a, c, d, f, h |
| 20 | c, d, f, g, h |
| 30 | c, d, f, g |

  - 从 T10 中去掉 "a (40)" 之后
    - T10 不再能满足 c3
      - 因为 "b (0)" 以及 "c (−20), d (−15), f (−10), h (5)"
    - 去掉 T10 后，也可剪去 T20 中的 "h"

b 的投影数据库

| TID | 事务 |
|---|---|
| ~~10~~ | ~~a, c, d, f, h~~ |
| 20 | c, d, f, g, h |
| 30 | c, d, f, g |

# 递归探索数据空间剪枝（续）




- 注：仅当在 b 的投影数据库中按 min-sup 剪去 "a" 后， c3 才能有效剪去 T10

# 模式空间与数据空间剪枝

| 问题 | 模式空间剪枝 | 数据空间剪枝 |
| --- | --- | --- |
| **去掉什么?** | 候选模式或分支 | 事务或投影记录 |
| **典型原因** | 任何扩展都无法满足 $c$ | 该记录无法支持满足约束的扩展 |
| **示例** | 支持度低于 $minsup$ | 可达最大利润低于 $v$ |
| **收益** | 候选更少 | 支持度计数代价更低 |

**强挖掘策略通常同时缩小两个空间，但每一步剪枝都需要各自的安全性论证。**

# 简洁性：同时剪枝数据空间与模式空间

- **简洁性：若约束 c 可通过直接操作数据来强制执行**
- 例 1： 寻找包含项 i 的模式
  - 只挖掘 i 投影数据库（数据空间剪枝）
- 例 2： 寻找不含项 i 的模式
  - 从数据库中删除 i 后再挖掘（模式空间剪枝）

- 例 3： c3: min(S.Price) ≤ v 是简洁的
  - 从合格的低价项出发，挖掘其投影数据库；高价项仍可加入
- 例 4： c4: sum(S.Price) ≥ v 不是简洁的
  - 无法事先确定，因为项集 S 的价格之和持续增大

# 简洁约束

## 可在挖掘前强制执行

- 包含项 i
- 排除项 i
- min(price) ≤ v，使用合格锚点与投影数据

## 无法事先判定

- sum(price) ≥ v
- 无有效排序时 average(profit) ≥ v
- 计数出现次数之前 support ≥ σ

**简洁性问的是：直接操作数据能否精确强制该约束。**

# 带约束的 FP-Growth： 将简洁约束深入下推

| 项 | 价格 |
|---|---|
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |
| 4 | 4 |
| 5 | 5 |

| TID | Items |
|---|---|
| 10 | 1 3 4 |
| 20 | 2 3 5 |
| 30 | 1 2 3 5 |
| 40 | 2 5 |

删除
不频繁
长度 1

| TID | Items |
|---|---|
| 10 | 1 3 |
| 20 | 2 3 5 |
| 30 | 1 2 3 5 |
| 40 | 2 5 |

Min_sup=2

约束： min{S.price} <= 2

无需对 3 或 5 投影

1- 投影数据库

| TID | Items |
|---|---|
| 10 | 3 |
| 30 | 2 3 5 |

2- 投影数据库

| TID | 项 |
|---|---|
| 20 | 3 5 |
| 30 | 1 3 5 |
| 40 | 5 |

# 不同种类的约束对应不同的剪枝策略

- 总之，约束可分为模式空间剪枝约束与数据空间剪枝约束

| 模式空间剪枝约束 | 数据空间剪枝约束 |
| --- | --- |
| ■ 反单调：若违反约束 c，可终止其后续挖掘<br>■ 单调：若已满足 c，无需再次检查 c<br>■ 可转换：若处理中能对项适当排序，则 c 可转换为单调或反单调<br>■ 简洁：若约束 c 可通过直接操作数据来强制执行 | ■ 数据简洁：可在模式挖掘初始阶段剪枝数据空间<br>■ 数据反单调：若事务 t 不满足 c，则可剪去 t 以减少数据处理开销 |

# 如何处理多个约束?

- 在模式挖掘中使用多个约束是有益的
- 但不同约束可能要求相互冲突的项排序
  - 若 c1 与 c2 的排序存在冲突
    - 尝试先排序数据并强制执行其中一个约束（选哪一个？）
    - 再在挖掘投影数据库时强制执行另一约束
- 例： c1: avg(S.profit) > 20 ， 以及 c2: avg(S.price) < 50
  - 假设 c1 的剪枝能力更强
    - 按利润降序排序并先使用 c1
  - 对每个投影数据库，按价格升序排列各项，并在挖掘中强制执行 c2

# 处理多个约束

- 估计哪个约束能去除搜索空间的最大部分
- 尽早应用简洁或强反单调约束
- 保持可转换约束所要求的项排序
- 仅当后续剪枝证明仍然成立时，才在投影数据库内重排
- 输出前验证全部约束，即使其中一些已用于早期剪枝

**最佳实施顺序取决于剪枝能力、计算代价与排序兼容性。**

# 第 1 部分综合：我们可以剪枝什么？

| 情形 | 安全操作 | 原因 |
| --- | --- | --- |
| **模式不满足反单调 $c$** | 停止扩展该分支 | 所有超集均不满足 |
| **模式满足单调 $c$** | 停止在超集上检查 $c$ | 所有超集均满足 |
| **约束是简洁的** | 先过滤或投影数据 | 直接实施是精确的 |
| **记录无法满足任何扩展** | 删除该记录 | 数据反单调性 |

**第 2 部分将模式表示从无序集合改为序列和图。**

# 附录

# 第 1 部分备用幻灯片

# 第 3 讲回顾： Apriori

| 阶段 | 操作 | 原因 |
| --- | --- | --- |
| 1. 连接 | 由 Fk−1 中的相容对生成 Ck | 只有频繁 (k−1)- 项集才能参与 |
| 2. 剪枝 | 若任一 (k−1)- 子集不在 Fk−1 中，则删除该候选 | 不频繁子集使每个超集都不频繁 |
| 3. 计数 | 扫描事务数据库，为幸存候选计数 | 支持度必须在数据上度量 |
| 4. 选择 | 当 support(X) ≥ minsup 时保留 X，以形成 Fk | 重复直到 Fk 为空 |

**Apriori 交替进行候选生成与数据库扫描；向下闭包提供剪枝保证。**

# 第 3 讲回顾： Apriori 的优化与替代方法

| 方法 | 变化之处 | 效率思路 |
| --- | --- | --- |
| **DHP** | Apriori 候选过滤 | 对 k- 项集哈希并剪掉低计数桶；缩减事务 |
| **Partition** | Apriori 数据库访问 | 挖掘局部频繁集，再在第二次扫描中验证其并集 |
| **ECLAT** | 垂直 TID 列表表示 | 在深度优先搜索中对 TID 集合求交 |
| **FP-growth** | 压缩前缀树表示 | 挖掘条件模式基，无需显式候选 |

**DHP 与 Partition 优化 Apriori；ECLAT 与 FP-growth 重组数据与搜索。**

# 第 1 部分学习目标

**更广的模式空间需要对所挖掘内容有更强控制。**

- 为层次化与异质项选择支持度阈值
- 区分稀有模式与负相关模式
- 将约束分为反单调、单调、可转换或简洁
- 判断约束能剪枝模式、数据，还是两者

# 统一支持度与逐层降低支持度

| 策略 | 示例 | 优点 | 风险 |
| --- | --- | --- | --- |
| **统一 minsup** | 每层均为 5% | 简单且可比 | 漏掉具体的低层项 |
| **逐层降低的 minsup** | 乳制品 5%；<br>牛奶种类 1% | 发现更细的模式 | 产生更多候选 |

**低层项分摊父项支持度，因此往往需要更低的阈值。**

# 稀有模式与负模式

| 性质 | 稀有模式 | 负模式 |
| --- | --- | --- |
| **主要问题** | 该模式是否很少出现? | A 与 B 的共现是否少于预期? |
| **所需边际** | 非核心 | A 与 B 各自都应出现得足够频繁 |
| **典型度量** | 带自定义阈值的支持度 | 一种零事务不变的相关度量 |
| **示例** | Rolex 购买 | 两种替代商品 |

**仅凭较低的联合支持度，无法区分稀有与负关联。**

# 数据挖掘中各类用户指定约束

- 知识类型约束 — 指定要挖掘的知识类型
  - 例如：分类、关联、聚类、离群点发现……
- 数据约束 — 使用类 SQL 查询
  - 例如：找出今年纽约商店中一起售出的商品
- 维度 / 层次约束 — 类似于关系数据库中的投影
  - 例如：与地区、价格、品牌、顾客类别相关
- 兴趣度约束 — 各类阈值
  - 例如：强规则： min_sup ≥ 0.02, min_conf ≥ 0.6, min_correlation ≥ 0.7
- 规则（或模式）约束 ← 本研究的重点
  - 例如：小额销售（price < $10）引发大额销售（sum > $200）

# Kulczynski 度量是零事务不变的

**Kulc 对两个条件共现概率取平均。**

1. 使用 count(A)=100 、 count(B)=100 与 count(A∪B)=1 。
2. P(B|A)=1/100=0.01 ，且 P(A|B)=1/100=0.01 。
3. Kulc(A,B)=[P(B|A)+P(A|B)]/2=(0.01+0.01)/2=0.01.
4. 加入零事务会改变 N ，但不改变这些条件概率。

**当负相关阈值 ε = 0.02 时， Kulc = 0.01 表示负关联。**

# 约束性质及其用途

| 性质 | 安全推断 | 主要用途 |
| --- | --- | --- |
| **反单调** | 失败在扩展下持续 | 剪枝模式分支 |
| **单调** | 满足在扩展下持续 | 停止重复检查 |
| **可转换** | 排序产生上述性质之一 | 实现前缀剪枝 |
| **简洁** | 直接选择即可实施该约束 | 缩小初始搜索空间 |
| **数据反单调** | 记录无法支持后续扩展 | 剪枝投影数据 |

**一个约束可能具有不止一种有用性质。**

# 三个约束的实例策略

**约束顺序是算法设计决策，而不仅是查询语言细节。**

1. 要求：支持度 ⩾ 2 。全程保持 Apriori 或 FP-growth 的支持度剪枝。
2. 要求： min(price) ≤ 50 。从符合条件的低价锚点与投影数据库出发。
3. 要求： average(profit) > 20 。在可行时采用经验证的利润降序增长顺序。
4. 输出前对幸存模式计数，并对照全部三个约束加以验证。

**先实施低代价、安全且剪枝力强的约束；推迟无法安全剪枝的高代价检查。**

# 第 1 部分检查

**问题：某模式违反反单调约束。挖掘算法可以安全地做什么？**

| | |
|---|---|
| A. 保留所有超集 | B. 剪枝每个超集 |
| C. 声明其为最大模式 | D. 降低 minsup |

**答案： B. 剪枝每个超集**

# 第 2 部分：序列模式与图模式

## 先顺序，后结构

- 挖掘序列模式
- 挖掘子图模式
- 小结

# 序列模式挖掘

- 什么样的模式是序列的?
- 序列：顺序真正重要
  - 不能交换序列中的两项却仍得到同一序列
- 例：英语是序列的：主语 → 动词 → 宾语

- 其他要点：
  - 序列模式使用事件顺序；精确时间戳与时间间隔是可选约束
  - 时间序列分析对按时间索引的测量建模，并常使用实际间隔

# 项集与序列问的问题不同

| 表示 | 等价? | 示例问题 |
| --- | --- | --- |
| 项集 {A,B} | {A,B} = {B,A} | 哪些项一起出现? |
| 序列 ⟨ A,B⟩ | ⟨A,B⟩ ≠ ⟨B,A⟩ | 哪个事件倾向于先发生? |
| 事件集 ⟨ (A B)⟩ | A 与 B 同属一个事件 | 哪些项出现在同一步? |

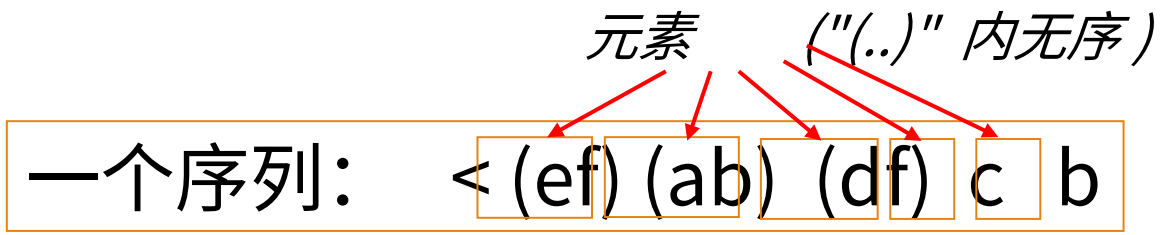
**序列模式挖掘保留事件之间的顺序，但不保留单个事件集内部的顺序。**

# 序列模式示例

- 序列模式挖掘的应用
  - **顾客购物：先买笔记本电脑，再买数码相机，然后买智能手机**
  - **医疗过程：看医生、取药、医生监测进展、医生据此应对 → 加药 / 减药**
  - **自然灾害：灾前、灾中、灾后**
  - **科学实验：步骤 1 、步骤 2 、步骤 3**
  - **市场事件：上涨、回调与复苏可能形成反复出现的有序模式**
  - **生物序列：改变核苷酸或氨基酸顺序会改变所得序列及其功能**

# 序列模式与序列模式挖掘

- **序列模式挖掘：给定一组序列，找出全部频繁子序列（即满足 min_sup 阈值）**




- 一个元素可以包含一组项（也称为事件）

* 元素内的项无序，并按字母顺序列出

一个序列数据库

| SID | 序列 |
|---|---|
| 10 | <a(abc)(ac)d(cf)> |
| 20 | <(ad)c(bc)(ae)> |
| 30 | <(ef)(ab)(df)cb> |
| 40 | <eg(af)cbc> |

# 阅读序列记号

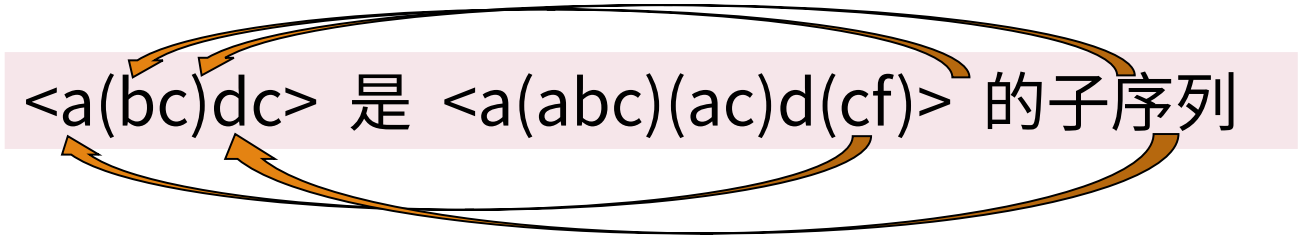
**括号将同一事件中观察到的项归为一组。**

1. 从左到右将 ⟨ a (bc) d⟩ 读作三个有序元素。
2. 第一个元素包含项 a 。
3. 第二个元素同时包含 b 和 c ；其内部顺序无关。
4. 第三个元素包含 d 。

**序列长度通常计项数，序列大小计元素数；应说明算法采用哪种约定。**

# 序列模式与序列模式挖掘

- **序列模式挖掘：给定一组序列，找出全部频繁子序列（即满足 min_sup 阈值）**




- Given support threshold *min_sup* = 2, <(ab)c> *is* a *sequential pattern*

一个序列数据库

| SID | 序列 |
|---|---|
| 10 | <a(abc)(ac)d(cf)> |
| 20 | <(ad)c(bc)(ae)> |
| 30 | <(ef)(ab)(df)cb> |
| 40 | <eg(af)cbc> |

# 子序列如何匹配

**匹配使用有序包含，而不是字符串相等。**

1. 目标模式：⟨ a (bc) d c⟩ 。
2. 数据库序列：⟨ a (abc) (ac) d (cf)⟩ 。
3. 将 a 匹配到第一个元素，将 (bc) 匹配到第二个元素，因为 $\{b,c\} \subseteq \{a,b,c\}$ 。
4. 将 d 匹配到第四个元素，将 c 匹配到第五个元素；未使用的元素可以跳过。

**子序列保持顺序，但不要求元素相邻，也不要求元素完全相等。**

# 序列模式的支持度

- 当 s 是某序列的子序列时，该序列支持模式 s
- 每个序列 ID 至多计入一次支持计数，即使 s 出现多次
- 支持计数 (s) = 包含 s 的序列 ID 数
- 相对支持度 (s) = 支持计数 (s) / 序列数

**频数统计的是支持该模式的序列数，而不是序列内部的嵌入次数。**

# 序列模式挖掘算法

- 算法要求：高效、可扩展、找出完整集合、纳入各类用户特定约束

- Apriori 性质仍然成立：若子序列 s1 不频繁，则 s1 的任何超序列都不可能频繁
- 代表性算法
  - GSP (Generalized Sequential Patterns)： Srikant & Agrawal， EDBT '96

# GSP：逐层序列挖掘

**Apriori 性质对子序列同样成立。**

1. 找出频繁的长度 1 序列。
2. 连接相容的频繁长度 k 序列，生成长度 (k+1) 候选。
3. 剪掉其必要子序列不频繁的候选。
4. 扫描序列数据库，统计支持度并保留频繁候选。
5. 重复，直到没有候选为止。

**GSP 把 Apriori 剪枝从项集推广到有序子序列，但仍需反复生成候选并扫描。**

GSP (Generalized Sequential Patterns)： Srikant & Agrawal @ EDBT'96)

# GSP：基于 Apriori 的序列模式挖掘

- 初始候选：全部 8 个单元素序列
  - <a>, <b>, <c>, <d>, <e>, <f>, <g>, <h>
- 扫描数据库一次，统计每个候选的支持度

| SID | 序列 |
|---|---|
| 10 | <(bd)cb(ac)> |
| 20 | <(bf)(ce)b(fg)> |
| 30 | <(ah)(bf)abf> |
| 40 | <(be)(ce)d> |
| 50 | <a(bd)bcb(ade)> |

*min_sup* = 2



| 候选 | 支持度 |
|---|---|
| <a> | 3 |
| <b> | 5 |
| <c> | 4 |
| <d> | 3 |
| <e> | 3 |
| <f> | 2 |
| <g> | 1 |
| <h> | 1 |

GSP (Generalized Sequential Patterns)：

# GSP：基于 Apriori 的序列模式挖掘

- 示例：生成长度 2 候选序列

*min_sup* = 2

| Cand. | sup |
|---|---|
| <a> | 3 |
| <b> | 5 |
| <c> | 4 |
| <d> | 3 |
| <e> | 3 |
| <f> | 2 |
| ~~<g>~~ | 1 |
| ~~<h>~~ | 1 |

单项 × 单项 – 合计：(6 * 6)

| | <a> | <b> | <c> | <d> | <e> | <f> |
|---|---|---|---|---|---|---|
| <a> | <aa> | <ab> | <ac> | <ad> | <ae> | <af> |
| <b> | <ba> | <bb> | <bc> | <bd> | <be> | <bf> |
| <c> | <ca> | <cb> | <cc> | <cd> | <ce> | <cf> |
| <d> | <da> | <db> | <dc> | <dd> | <de> | <df> |
| <e> | <ea> | <eb> | <ec> | <ed> | <ee> | <ef> |
| <f> | <fa> | <fb> | <fc> | <fd> | <fe> | <ff> |

集合（无序）– 合计：(6*5) / 2

| | <a> | <b> | <c> | <d> | <e> | <f> |
|---|---|---|---|---|---|---|
| <a> | | <(ab)> | <(ac)> | <(ad)> | <(ae)> | <(af)> |
| <b> | | | <(bc)> | <(bd)> | <(be)> | <(bf)> |
| <c> | | | | <(cd)> | <(ce)> | <(cf)> |
| <d> | | | | | <(de)> | <(df)> |
| <e> | | | | | | <(ef)> |
| | | | | | | |

Apriori 剪枝

- 无剪枝（含 g 和 h）：

  8*8 + 8*7/2 = 92

  长度为 2 的候选

- 有剪枝：

  6*6 + 6*5/2 = 51

  长度为 2 的候选

GSP (Generalized Sequential Patterns)： Srikant & Agrawal @ EDBT'96)

# 三类长度 2 候选

| 候选形式 | 含义 | m 个频繁项时的数量 |
| --- | --- | --- |
| ⟨a b⟩ | a 出现在 b 之前 | $m^2$，包括重复项 |
| ⟨b a⟩ | b 出现在 a 之前 | 已包含在 $m^2$ 中 |
| ⟨(a b)⟩ | a 与 b 同时出现 | $m(m-1)/2$ |

**当 m=6 时，GSP 生成 $6^2 + 6\times5/2 = 51$ 个长度 2 候选。**

# GSP 挖掘与剪枝



| 长度 | |
|---|---|
| 5 | 5 扫 : 1 候选 , 1 个长 5 模式 <(bd)cba> |
| 4 | 4 扫 : 8 候选 , 7 个长 4 模式 <abba> <(bd)bc> … |
| 3 | 3 扫 : 46 候选 , 20 个长 3, 20 不在库 <abb> <aab> <aba> <baa> <bab> … |
| 2 | 2 扫 : 51 候选 , 19 个长 2, 10 不在库 <aa> <ab> … <af> <ba> <bb> … <ff> <(ab)> … <(ef)> |
| 1 | 1 扫 : 8 候选 , 6 个长 1 模式 <a> <b> <c> <d> <e> <f> ~~<g> <h>~~ |

- 去掉不在库或支持度 < min_sup 的候选

$6*6 + 6*5/2 = 51$

GSP 算法

- 从 k = 1 开始重复，直到 k <= length
  - 扫描数据库，找出“长度为 k”的频繁序列
  - 用 Apriori 从“长度为 k”的频繁序列生成“长度为 (k+1)”的候选序列
  - 令 k = k+1
- 直到找不到频繁序列或候选

*min_sup* = 2

| SID | 序列 |
|---|---|
| 10 | <(bd)cb(ac)> |
| 20 | <(bf)(ce)b(fg)> |
| 30 | <(ah)(bf)abf> |
| 40 | <(be)(ce)d> |
| 50 | <a(bd)bcb(ade)> |

# GSP 为何代价变高

## 计算

- 每个序列长度扫描一次数据库
- 早期层次候选集很大
- 计数时子序列匹配代价高

## 搜索浪费

- 许多候选在数据库中从未出现
- 长模式产生大量重叠候选
- 稀疏序列几乎无法复用候选

**模式增长方法在投影数据库内工作，从而避免生成全局候选。**

# PrefixSpan： 按前缀增长模式

- 选择一个频繁前缀，例如⟨ a⟩
- 构建前缀投影数据库，只保留该前缀出现之后的后缀
- 在该较小数据库中统计局部频繁的扩展项
- 扩展该前缀并递归，而不生成全局候选集

**投影把一个大搜索问题转化为若干更小的条件问题。**

# 一个简单的前缀投影数据库

| SID | 原始序列 | 前缀 ⟨ a⟩ 之后的后缀 |
| --- | --- | --- |
| S1 | ⟨a b c⟩ | ⟨b c⟩ |
| S2 | ⟨a c b⟩ | ⟨c b⟩ |
| S3 | ⟨b a c⟩ | ⟨c⟩ |

**在 ⟨ a⟩ 投影数据库中， b 出现在两个后缀中， c 出现在三个后缀中。**

# PrefixSpan 中的前缀增长

**每次递归调用只挖掘与其前缀相容的序列。**

1. 从前缀 ⟨ a⟩ 及其投影数据库开始。
2. 若 minsup 计数为 2，则 b 和 c 均可作为扩展。
3. 生成 ⟨ a b⟩ 与 ⟨ a c⟩，并为每个前缀构建新的投影数据库。
4. 继续，直到不再有局部频繁扩展。

**PrefixSpan 仍依赖支持度的反单调性；它改变的是搜索空间的表示与探索方式。**

# GSP 与 PrefixSpan

| 方面 | GSP | PrefixSpan |
| --- | --- | --- |
| 搜索 | 广度优先 | 深度优先模式增长 |
| 候选 | 显式全局候选 | 前缀扩展 |
| 数据库访问 | 反复全库扫描 | 递归投影数据库 |
| 主要代价 | 候选生成与匹配 | 投影规模与递归 |

**两种方法并非在每个数据集上都占优；序列密度与模式长度决定这一权衡。**

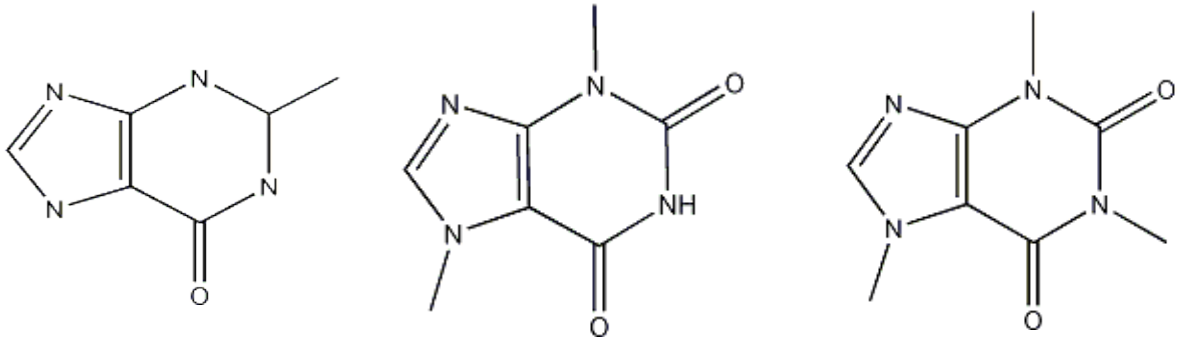
# 模式挖掘：高级方法

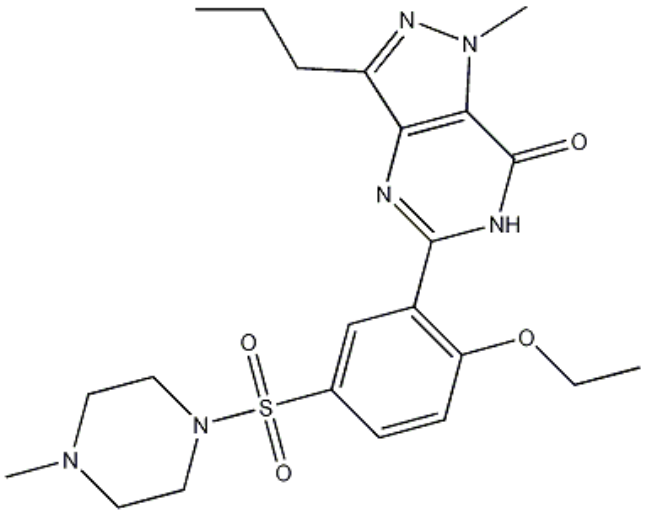
- 挖掘多层与多维模式
- 基于约束的模式挖掘
- 挖掘序列模式
- 挖掘子图模式
- 小结



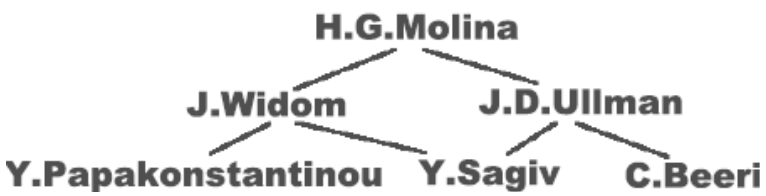
# 什么是图模式挖掘?

- 化学信息学:
  - 挖掘频繁化合物结构



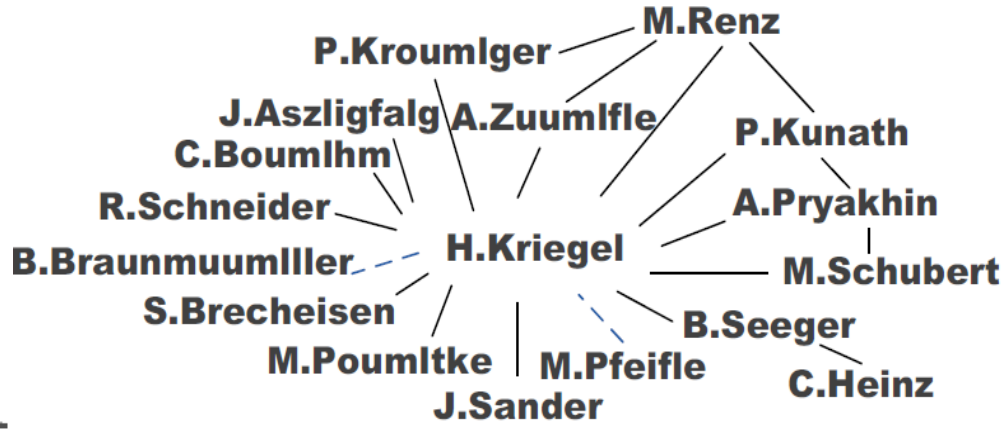


- 社交网络、网络社区、推文……
  - 寻找频繁的科研合作子图









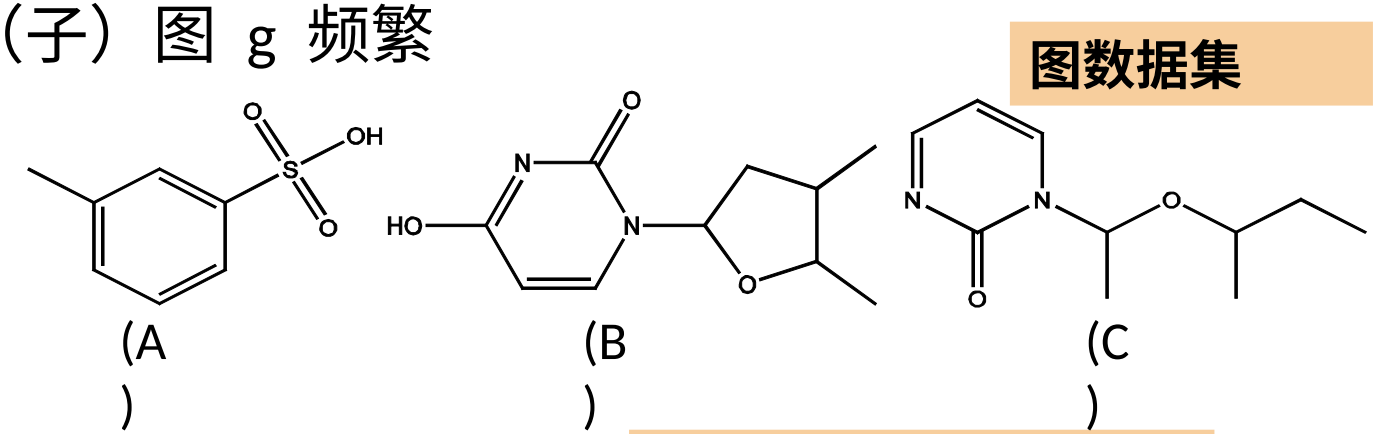
# 带标号图模式

| 术语 | 含义 | 示例 |
| --- | --- | --- |
| 顶点标号 | 实体类型 | 碳原子；用户角色 |
| 边标号 | 关系类型 | 化学键；关注 |
| 子图 | 选定顶点与边，并保持标号 | 官能团 |
| 嵌入 | 模式到图的一次映射 | 一次出现位置 |

**图模式挖掘必须同时处理组合增长与图同构。**

# 频繁（子）图模式

- 给定带标号图数据集 D = {G1, G2, …, Gn)，子图 g 的支持图集为 Dg = {Gi | g ⊆ Gi, Gi ∈D}
  - support(*g*) = $|D_g|/|D|$
- 若 support(g) ≥ min_sup，则（子）图 g 频繁
- 例：化学结构
- 另一种做法：
  - 从单个大图或网络中挖掘频繁子图模式



# 支持度按图计数

**不要把嵌入次数与图级支持度混淆。**

1. 设 D 含三个带标号图 G1、G2 和 G3。
2. 模式 g 在 G1 中出现两次，在 G2 中出现一次，在 G3 中不出现。
3. 支持图集为 Dg={G1,G2}。
4. support count(g)=2，相对 support(g)=2/3。

**在图事务数据库中，同一图内的多次嵌入仍只计为一个支持图。**

# 图的支持度计数为何困难

## 对项集

- 包含判定只是简单的子集检验
- 项的规范次序立即可得
- 支持度计数可用哈希或集合交

## 对图

- 包含判定需要保标号的节点映射
- 同一图有许多等价画法与遍历
- 子图同构主导计数代价

**图挖掘既需要剪枝，也需要规范表示。**

# 快速检查：图支持度

**问题：某子图在 G1 中出现五次，在 G2 中一次，在 G3 中零次。其支持度计数是多少?**

| | |
|---|---|
| A. 1 | B. 2 |
| C. 5 | D. 6 |

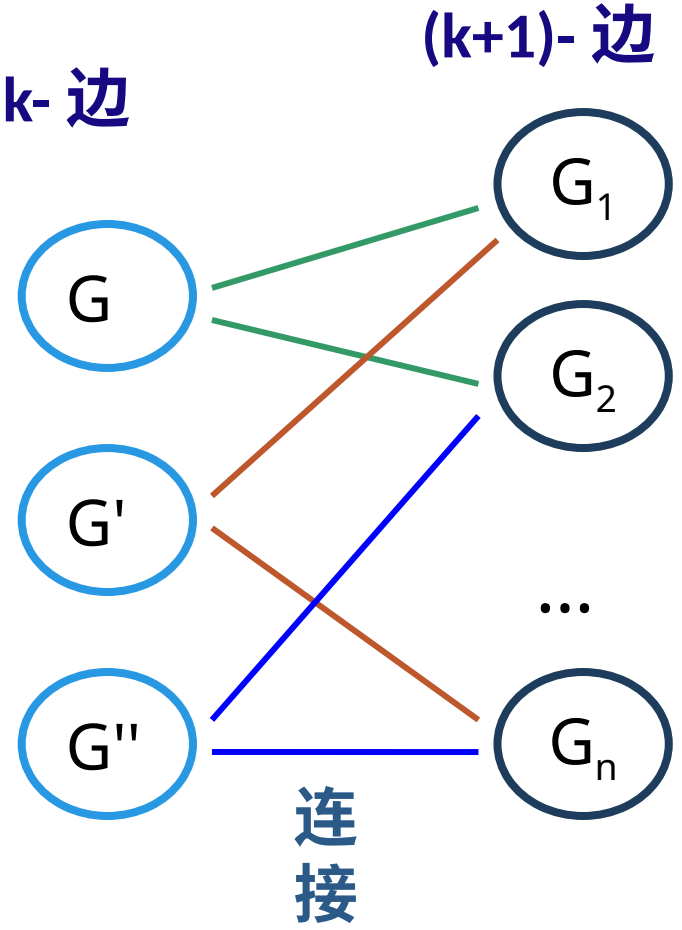
**答案： B. 2 个支持图**

# 图模式挖掘的应用

- 生物信息学
  - 基因网络、蛋白质相互作用、代谢通路
- 化学信息学：挖掘化合物结构
- 社交网络、网络社区、推文……
- 手机网络、计算机网络……
- Web 图、 XML 结构、语义网、信息网络
- 软件工程：程序执行流分析
- 图分类、聚类、压缩、比较与相关分析的基本构件
- 图索引与图相似性搜索

# 基于 Apriori 的方法

- Apriori 性质：若大小为 k 的子图频繁，则其每个子图都频繁；一个不频繁子图即可剪枝该候选
- 连接相容的频繁 k- 子图，生成大小为 (k+1) 的候选
- 迭代挖掘过程：
  - 候选生成 → 候选剪枝 → 支持度计数 → 候选消除



# Apriori 式图挖掘

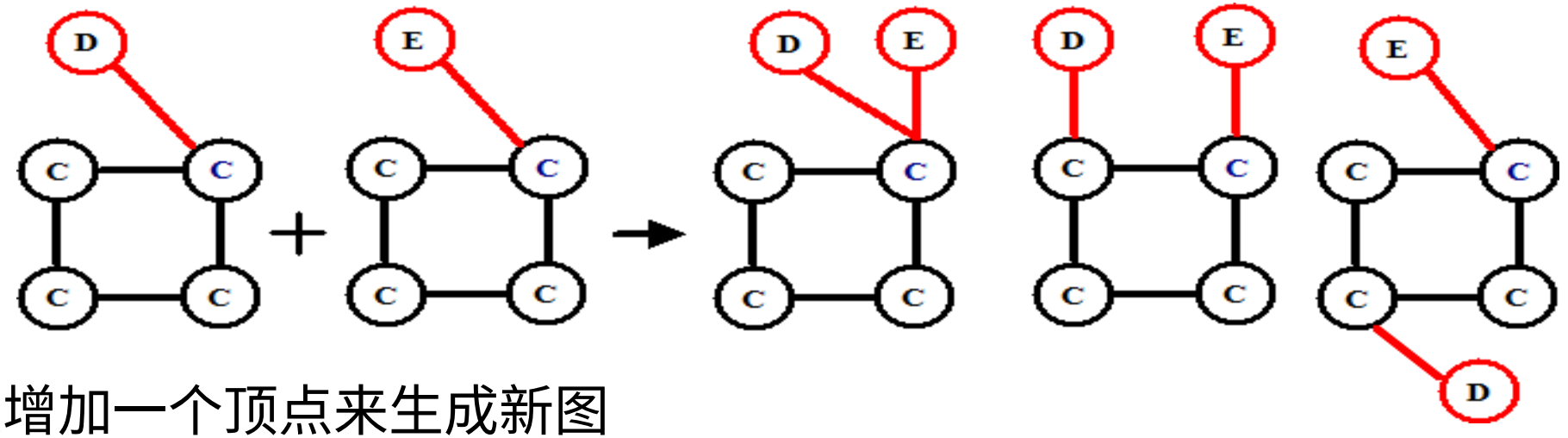
**图的连接比项集连接更复杂，因为共享子图可以有多种映射。**

1. 连接相容的频繁 k- 子图，生成大小为 (k+1) 的候选。
2. 对候选做规范化并去除重复。
3. 若任一必需的 k- 子图不频繁，则剪枝该候选。
4. 检验哪些数据库图包含该候选，以计数支持度。

**频繁蕴含每个子图都频繁；反之不成立。**

# 候选生成：顶点增长与边增长

- 方法：对大小为 k 的图做广度优先搜索与 Apriori 式连接
  - 生成大小为 (k+1) 的候选图时存在多种可能




- 每次增加一个顶点来生成新图
  - **AGM**：基于 Apriori 的图挖掘 (Inokuchi, Washio, & Motoda, PKDD’ 00)
- 每次增加一条边来生成新图
  - **FSG**：频繁子图发现 (Kuramochi & Karypis, ICDM’ 01)
- 边增长通常比顶点增长更能减少候选重复

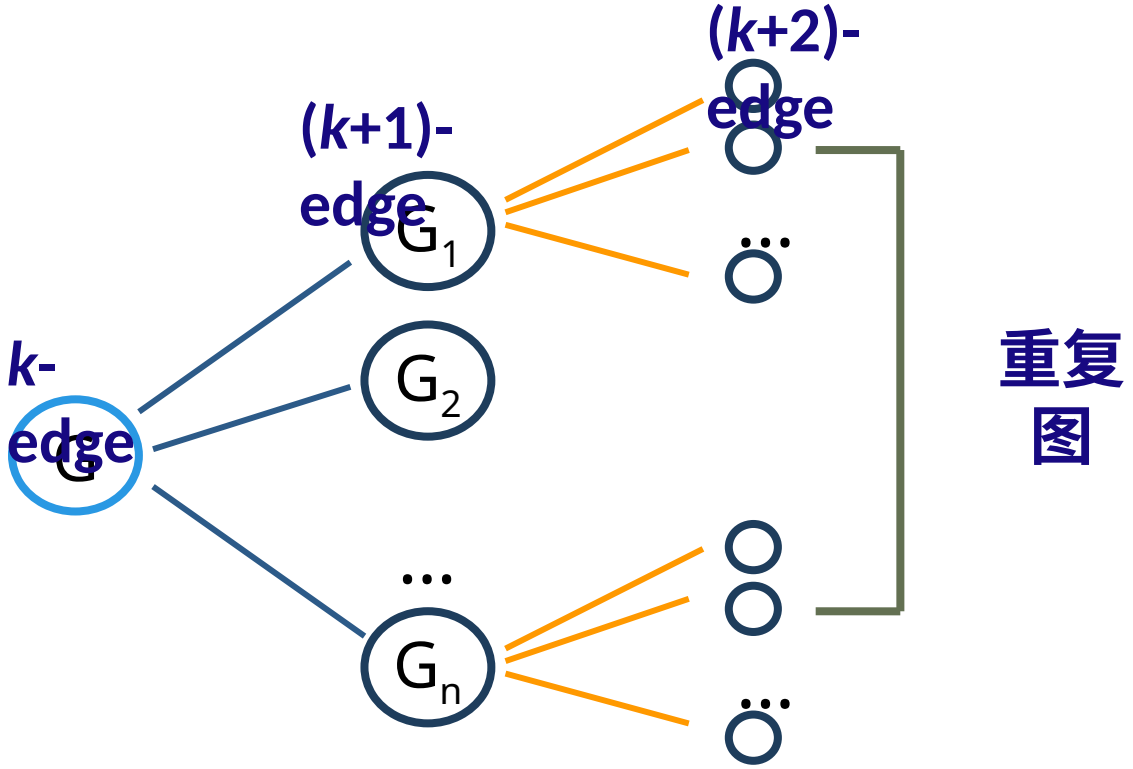
# 重复候选问题

- 同一无标号结构可用不同顶点编号画出
- 不同增长历史可产生同构候选
- 朴素去重需要反复做图同构检验
- 规范编码为每个图同构类指定一个代表

# 模式增长方法

- **子图从 k- 边经深度优先增长到 (k+1)- 边，再到 (k+2)- 边子图**

- 主要挑战
  - 生成大量重复子图
- 解决问题的主要思想
  - 定义子图的生成顺序
  - **DFS 生成树：用深度优先搜索把图展平为序列**
    - gSpan (Yan & Han, ICDM'02)




**gSpan 只增长规范 DFS 编码，从而避免大多数重复。**

# gSpan 中的规范 DFS 编码

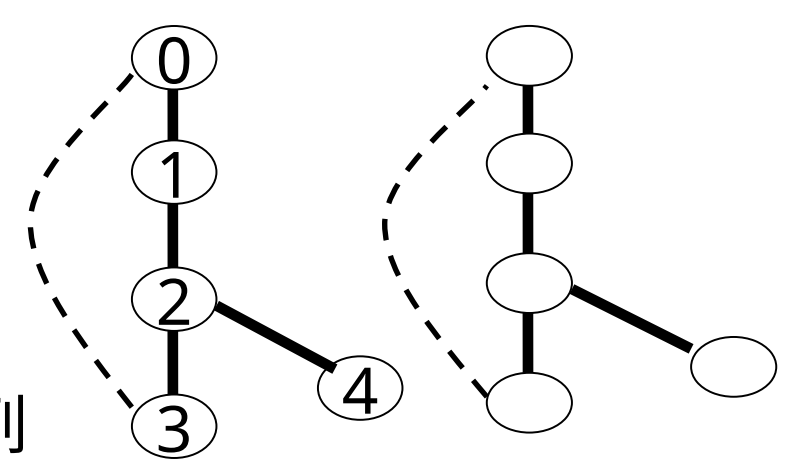
**该编码表示结构、标号与遍历顺序。**

1. 运行深度优先搜索，记录每条遍历边的端点下标与标号。
2. 一幅图可有许多合法 DFS 编码，因为遍历起点与分支可以不同。
3. 按字典序比较这些编码，保留最小 DFS 编码。
4. 挖掘过程中只有最小编码代表该图。

**规范编码把图等价检验转化为增长过程中一致的序比较。**

# gSpan： 按序增长图模式

- **子图模式增长中的最右路径扩展**
  - **最右路径： DFS 树中从根到最近发现顶点的路径**
  - 减少重复子图的生成
- **完备性：用最右路径扩展枚举图是完备的**
- DFS 编码：用深度优先搜索把图展平为序列




$e_0$: (0,1)

$e_1$: (1,2)

$e_2$: (2,3)

$e_3$: (3,0)

$e_4$: (2,4)

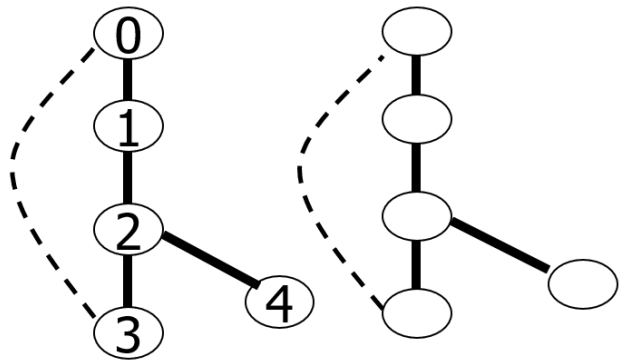
# 最右路径扩展

## 前向扩展

- 从当前最右路径上的一个顶点出发
- 增加一个新顶点与一条新边
- 新顶点成为最新的 DFS 顶点

## 后向扩展

- 从当前最右顶点出发
- 连接到最右路径上的一个祖先
- 不增加顶点而闭合一个环




$e_0$: (0,1)

$e_1$: (1,2)

$e_2$: (2,3)

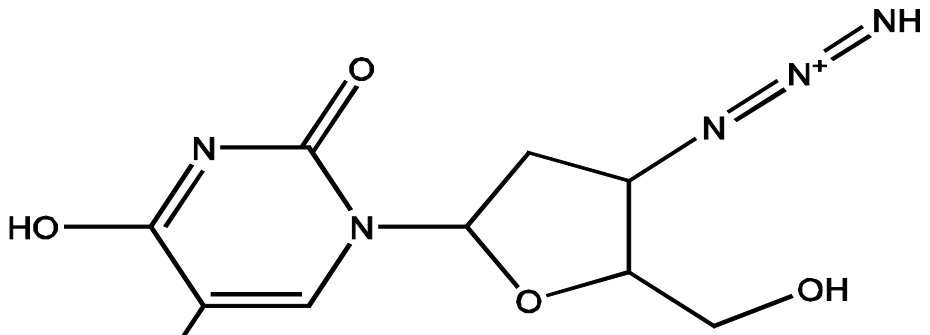
$e_3$: (3,0)

$e_4$: (2,4)

| | |
|---|---|
| $e_0 : (0, 1)$ | Forward |
| $e_1 : (1, 2)$ | Forward |
| $e_2 : (2, 3)$ | Forward |
| $e_3 : (3, 0)$ | Backward |
| $e_4 : (2, 4)$ | Forward |

# 挖掘闭图模式

- 最多 $2^n$ 个边子集 → 闭频繁子图
- 频繁图 G 是闭的，若不存在与 G 支持度相同的超图




闭意味着不存在支持图集合相同的真频繁超图

- *无损压缩：不包含非闭图，但仍保证挖掘结果完备*
- 算法 CloseGraph： 直接挖掘闭图模式

# 闭频繁图

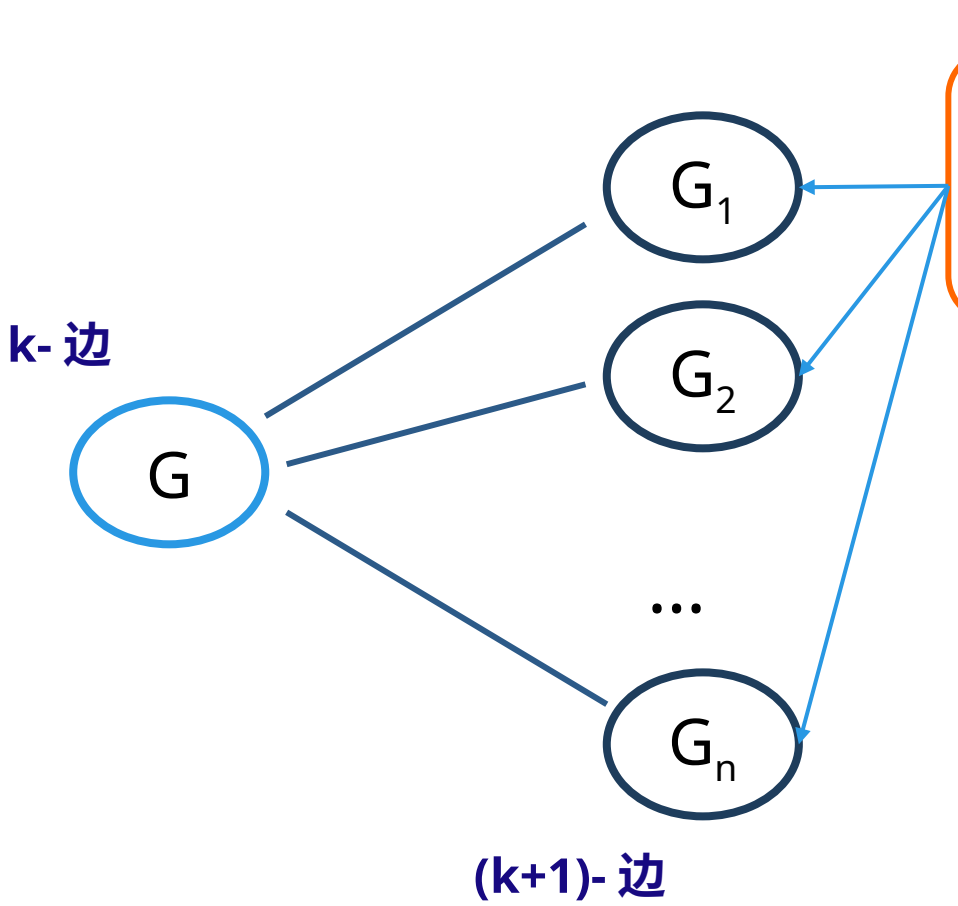
**定义与闭项集同一思想，只是用图包含代替集合包含。**

1. 图 G 频繁，并由 {G1,G2,G3} 支持。
2. 真超图 H 同样由 {G1,G2,G3} 支持。
3. 因此 G 与 H 的支持度相等。
4. G 不是闭的，因为 H 增加了结构且未失去任何支持图。

**闭图模式去除支持度等价的冗余，同时保留精确的支持度信息。**

# CloseGraph： 直接挖掘闭图模式

- CloseGraph： 通过扩展 gSpan 挖掘闭图模式 (Yan & Han, KDD'03)




- 设 G 与 G1 频繁，且 G 是 G1 的子图
- 若 G 的每次出现都能扩展到 G1 而不降低支持度， CloseGraph 可剪掉从 G 出发的冗余增长

# CloseGraph 提前终止

- 设 G 的每次出现都能扩展到更大的图 H
- 则 G 与 H 共享同一支持图集合
- G 不可能是闭的，因为 H 是支持度相等的真超图
- 从更大的代表继续增长，并在闭包条件安全时剪掉 G 的冗余后代

**剪枝判断取决于出现级等价，而不只是数值支持度相等。**

# 频繁图挖掘算法

| 方法 | 搜索 | 重复控制 | 输出 |
| --- | --- | --- | --- |
| **AGM / FSG** | 广度优先候选 | 连接后规范化 | 全部频繁图 |
| **gSpan** | 深度优先模式增长 | 最小 DFS 编码 | 全部频繁图 |
| **CloseGraph** | 基于 gSpan 的闭增长 | 基于闭包的剪枝 | 闭频繁图 |

**这些方法的主要差别在于枚举顺序、重复处理与输出压缩。**

# 第 4 讲综合

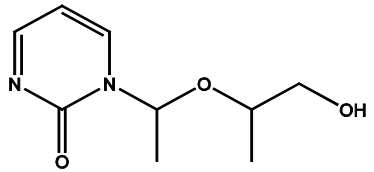
| 数据与模式 | 主要挑战 | 代表性思想 |
| --- | --- | --- |
| 层次 / 多维项 | 阈值与冗余 | 层次感知的支持度与祖先过滤 |
| 基于约束的项集 | 大量无关搜索空间 | 把安全约束推进挖掘 |
| 序列 | 顺序与大量候选 | GSP 或前缀投影增长 |
| 图 | 同构与重复 | 规范 DFS 增长与闭模式 |

**在全部四种设定中，表示方式与反单调性决定哪些可以安全剪枝。**

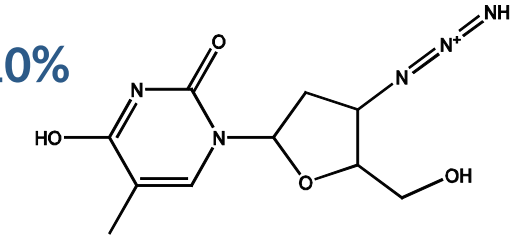
# 实验与性能比较

- 来自 NCI/NIH 的 AIDS 抗病毒筛选化合物数据集
- 该数据集包含 43,905 个化合物
- 发现的模式：最小支持度越小，发现的子图模式越大、越有意义

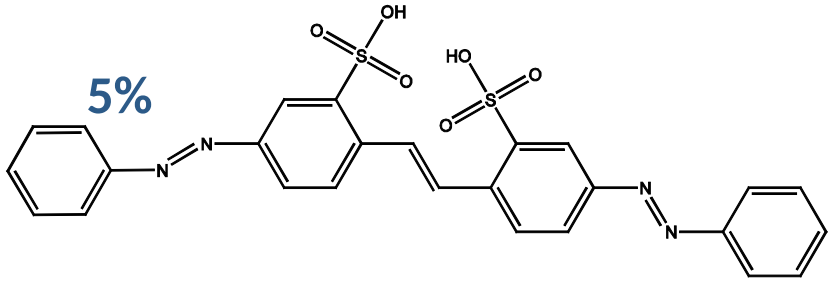
20%



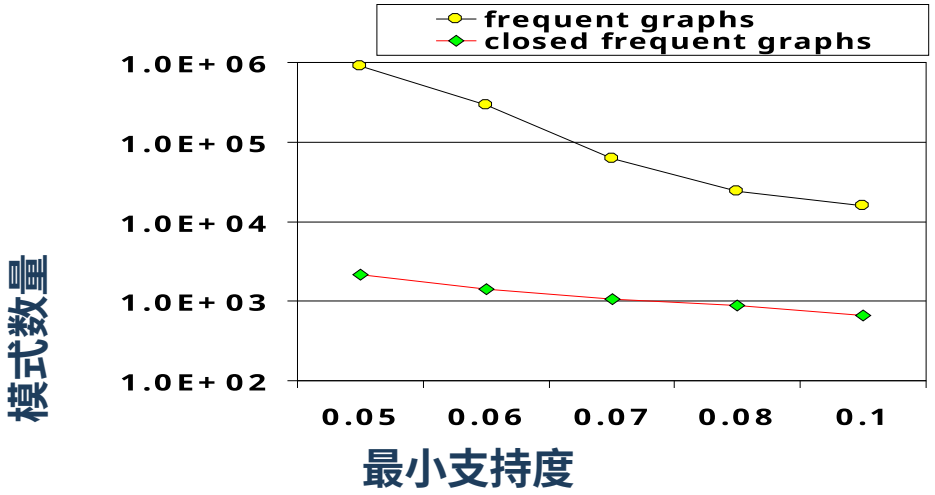
10%



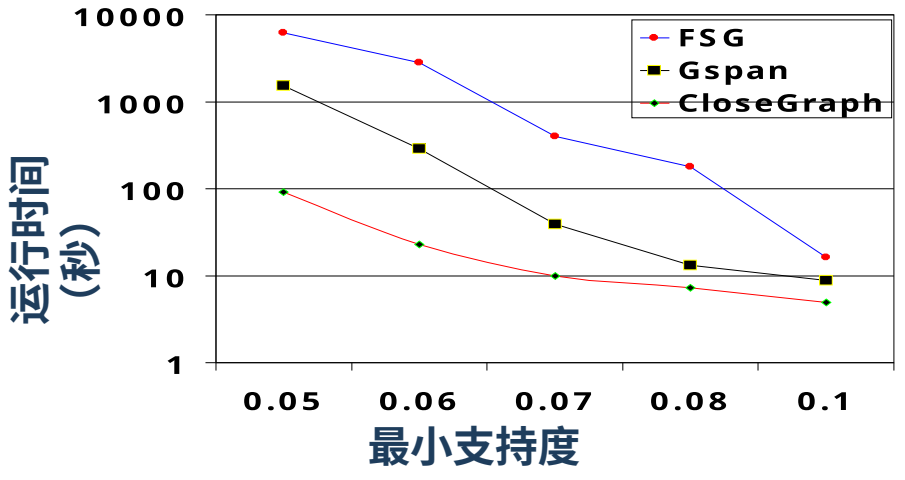
5%



**# 模式数：频繁与闭**

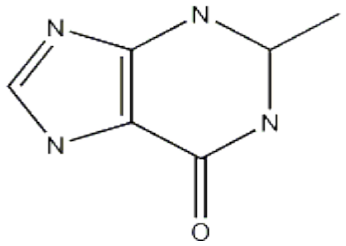


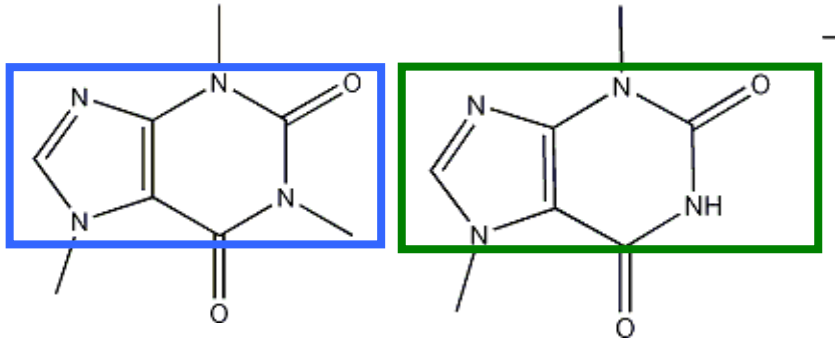

**运行时间：频繁与闭**



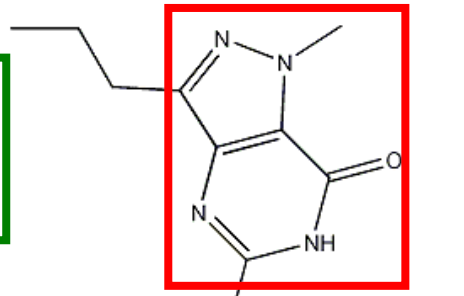
# 应用：支持子结构相似性搜索

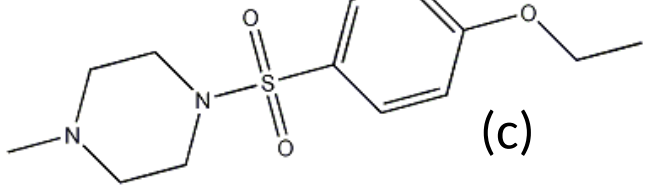
- 在图数据库中查找包含与给定查询图相似子结构的图
- 例．数据：化合物数据库
  - 查询图 q：





(a)　(b)





(c)

- 如何高效做相似性搜索?
  - 顺序扫描 + 计算子图相似度——代价过高!
  - 建立图索引以支持近似搜索?
    - 需要爆炸式数量的子图才能覆盖所有相似子图!
- 一个精巧的方案（Yan, Yu, & Han, SIGMOD'05）：
  - 保留图索引结构，但在查询空间中选择特征

# 最终快速检查

**问题：下列哪项陈述正确？**

| | |
|---|---|
| A. 顺序在序列中从不重要 | B. 含不频繁子图的候选可以被剪枝 |
| C. 每个闭图都是极大的 | D. Lift 是零不变的 |

**答案： B**